**Sequential baseline matrix multiplication**

Code comparison:

- This part of the project was the sequential baseline:  just the plain naive triple nested loop running on Zeus-2 with no optimizations. I compiled with -O0, seeded with srand(10), and used clock_gettime(CLOCK_MONOTONIC) to time only the matmul itself, not the setup. I also added a checksum after the multiplication to verify correctness, which came out to 21733579741 consistently across every run. Group 8's baseline (mem_heir.c) does the same thing at a similar high level, same algorithm, same seed, same timing approach. The main difference is how memory is set up. I used static 2D arrays so all the matrix data sits in one contiguous block of memory. They used malloc to allocate each row separately, which means the rows can end up scattered around in memory. That can hurt cache performance since accessing column j of B already jumps around in memory, and scattered rows make it worse.

Comparing Results:

- Results wise both implementations came out pretty similar. My average was 7419 ms across 5 runs with Zeus at 99.9% idle. Group 8's mem_heir.c produced times in the same ballpark. The small differences between runs on either side are just normal noise from running on a shared cluster. One thing worth noting is that our checksum functions work differently. Mine is a simple double loop over C that adds everything up. Group 8's checksum has a triple nested loop that ends up counting each element 1024 times, so the numbers won't match even if the math is correct. It doesn't affect the actual multiplication but means you can't directly compare the two checksum values. Both baselines land in the same 7.4 to 7.6 second range, which makes sense since it's the same algorithm. This number is both teams reference point that will be compared against.

My slides:

# Sequential Baseline Matrix Multiplication (Joel Haile)

**What's been implemented:**

- Ran on Zeus-2
- Matrix size: 1024 x 1024
- Compiled with -O0

**Implementation: Baseline**
**Average time: 7419 ms (7.4 seconds)**

- Seeded with srand(10)
- Added checksum verification (=21733579741)
- CPU load verified before runs (99.9% idle)

| Test # | Time (ms) | Checksum |
|---|---|---|
| 1 | 7392.927 | 21733579741 |
| 2 | 7398.796 | 21733579741 |
| 3 | 7416.660 | 21733579741 |
| 4 | 7429.402 | 21733579741 |
| 5 | 7458.007 | 21733579741 |
| **Average:** | **7419.151** | **21733579741** |

Their slides:

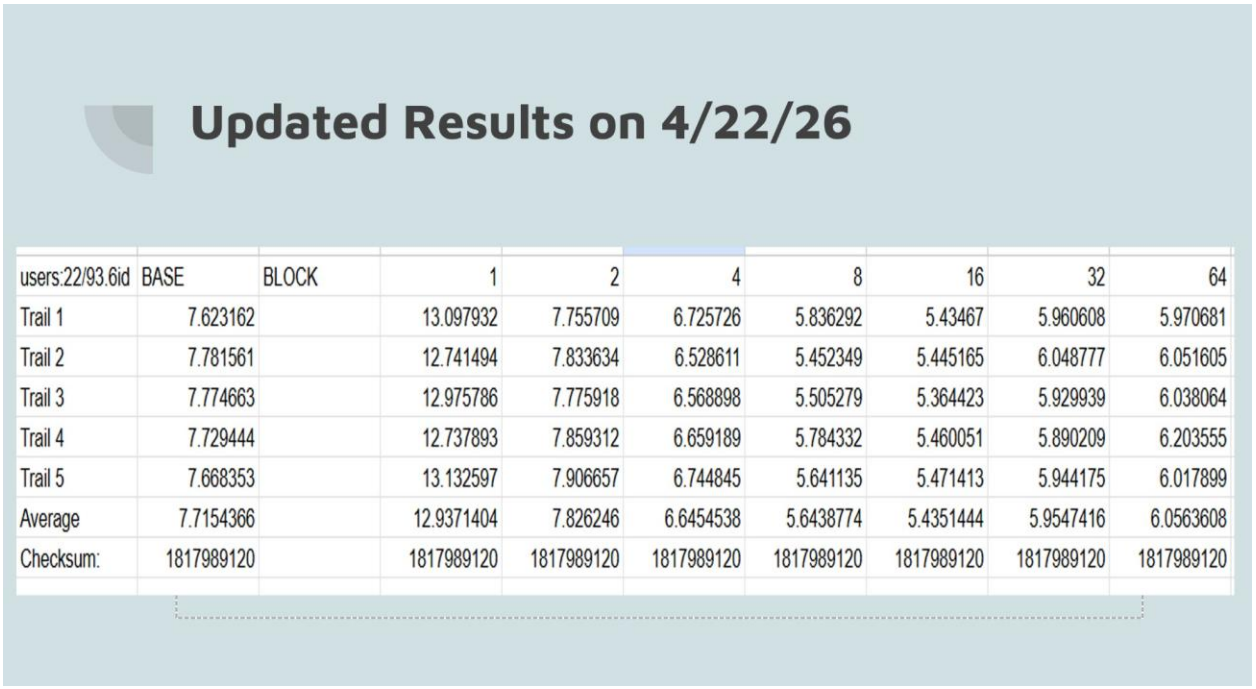
# Initial results from Section 1

```
[ahallid@zeus-2 CS465]$ w
 14:41:55 up 88 days,  1:35, 39 users,  load average: 6.57, 9.71, 10.33
USER     TTY      FROM             LOGIN@   IDLE   JCPU   PCPU WHAT
```

Graphs and tables

```
[ahallid@zeus-2 CS465]$ ./testing
Elapsed time: 10.530330 seconds
[ahallid@zeus-2 CS465]$ ./testing
Elapsed time: 10.726993 seconds
[ahallid@zeus-2 CS465]$
```

```
[ahallid@zeus-2 CS465]$ w
 16:49:26 up 85 days,  3:42, 24 users,  load average: 0.23, 0.52, 0.57
USER     TTY      FROM             LOGIN@   IDLE   JCPU   PCPU WHAT
mrodri74 pts/0    10.172.30.165    16:29    7:10   0.25s  0.22s vim arty.c
aortiz31 pts/1    10.151.145.154   14:32    2:10m  0.09s  0.04s vim arty.c
aortiz31 pts/2    10.151.145.154   16:11   37:26   0.03s  0.03s -bash
tvirk    pts/3    10.172.30.119    15:33    1:42   1.70s  1.67s vim arty.c
tbowers6 pts/6    10.172.30.156    16:34    6.00s  0.78s  0.75s vim arty.c
yliu80   pts/8    10.172.30.160    14:17    2:03m  0.04s  0.04s -bash
achoi30  pts/9    10.172.30.179    16:04   14.00s  0.30s  0.25s vim arty.c
dbell26  pts/10   10.172.30.122    15:00    1:29   0.29s  0.18s vim ./src/a
zmatthe2 pts/11   10.166.128.66    16:19   10:34   0.05s  0.05s -bash
istkhapi pts/12   10.172.30.87     15:02    1:47m  0.02s  0.02s -bash
aortiz31 pts/13   10.151.145.154   15:45   46.00s  1.08s  0.98s vim arty.c
aboese   pts/14   10.151.184.156   14:39    1:59m  0.32s  0.29s vim arty.c
aboese   pts/17   10.151.184.156   14:39    2:08m  0.03s  0.03s -bash
ahallid  pts/19   10.151.203.198   16:21    5.00s  0.08s  0.01s w
akhadem  pts/16   10.166.183.191   16:48    3.00s  0.02s  0.02s -bash
lkillea  pts/21   10.172.30.173    15:42   30.00s  0.76s  0.70s vim HW3-G01
anazaro  pts/22   10.172.30.81     15:48   30:23   0.02s  0.00s ./arty
habugha  pts/23   10.151.147.99    16:34   14:31   0.03s  0.03s -bash
ashahza5 pts/25   10.151.181.179   15:56   52:18   0.14s  0.11s vim getpid.
frodri3  pts/27   10.151.146.87    16:21   35.00s  0.54s  0.51s vim arty.c
root     pts/30   10.48.179.62     10:48    6:00m  0.03s  0.03s -bash
yzhong   pts/32   10.151.189.40    13:49   22:37   0.61s  0.61s -bash
[ahallid@zeus-2 CS465]$ ./testing
Elapsed time: 7.227049 seconds
```




| users:22/93.6id | BASE | BLOCK | 1 | 2 | 4 | 8 | 16 | 32 | 64 |
|---|---|---|---|---|---|---|---|---|---|
| Trail 1 | 7.623162 | | 13.097932 | 7.755709 | 6.725726 | 5.836292 | 5.43467 | 5.960608 | 5.970681 |
| Trail 2 | 7.781561 | | 12.741494 | 7.833634 | 6.528611 | 5.452349 | 5.445165 | 6.048777 | 6.051605 |
| Trail 3 | 7.774663 | | 12.975786 | 7.775918 | 6.568898 | 5.505279 | 5.364423 | 5.929939 | 6.038064 |
| Trail 4 | 7.729444 | | 12.737893 | 7.859312 | 6.659189 | 5.784332 | 5.460051 | 5.890209 | 6.203555 |
| Trail 5 | 7.668353 | | 13.132597 | 7.906657 | 6.744845 | 5.641135 | 5.471413 | 5.944175 | 6.017899 |
| Average | 7.7154366 | | 12.9371404 | 7.826246 | 6.6454538 | 5.6438774 | 5.4351444 | 5.9547416 | 6.0563608 |
| Checksum: | 1817989120 | | 1817989120 | 1817989120 | 1817989120 | 1817989120 | 1817989120 | 1817989120 | 1817989120 |

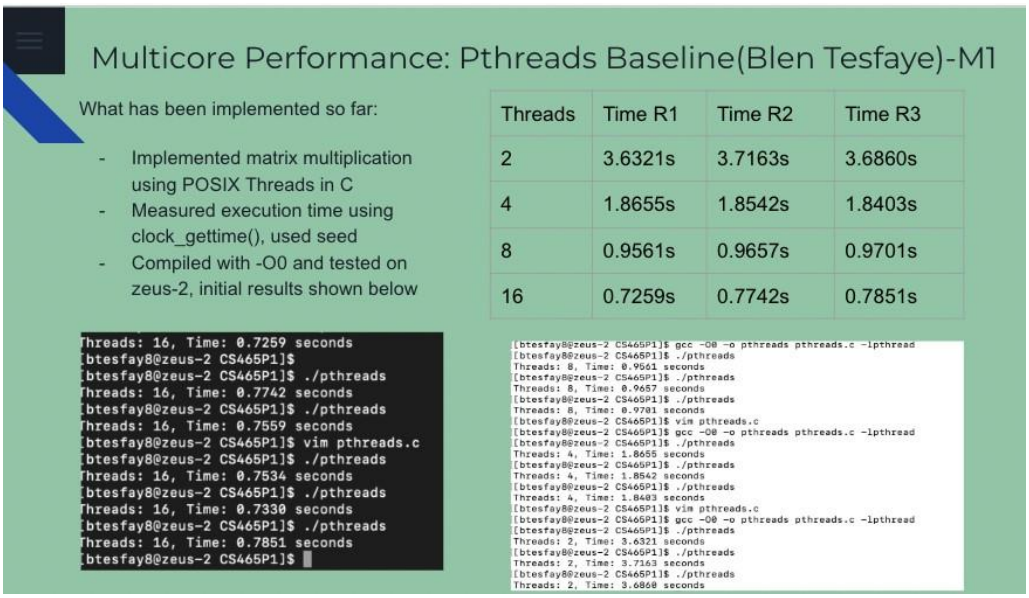
**Multicore Performance: Pthreads**

- When comparing our code to the other group's, a few things stand out. The biggest difference is how we set up the matrices. I used simple static arrays, which keeps everything in one place in memory, making it faster to access. They used malloc to allocate each row separately, which can slow things down because the rows end up spread out in memory. One thing they did that I didn't was save A[i][k] into a local variable inside the tiled loop so it doesn't have to keep grabbing that value from memory over and over again; that's a smart optimization. They also made their code easier to test by passing thread count and tile size as command-line arguments, so they didn't have to recompile every time they wanted to try a different value like I did. For checking correctness, they saved the checksum to a file and automatically compared it each run, while I just printed and checked mine manually by looking at the output. At a high level, like splitting the matrix rows across threads, were similar, but these little differences in how the memory is set up and how the code is organized explains why one performed better than the other.

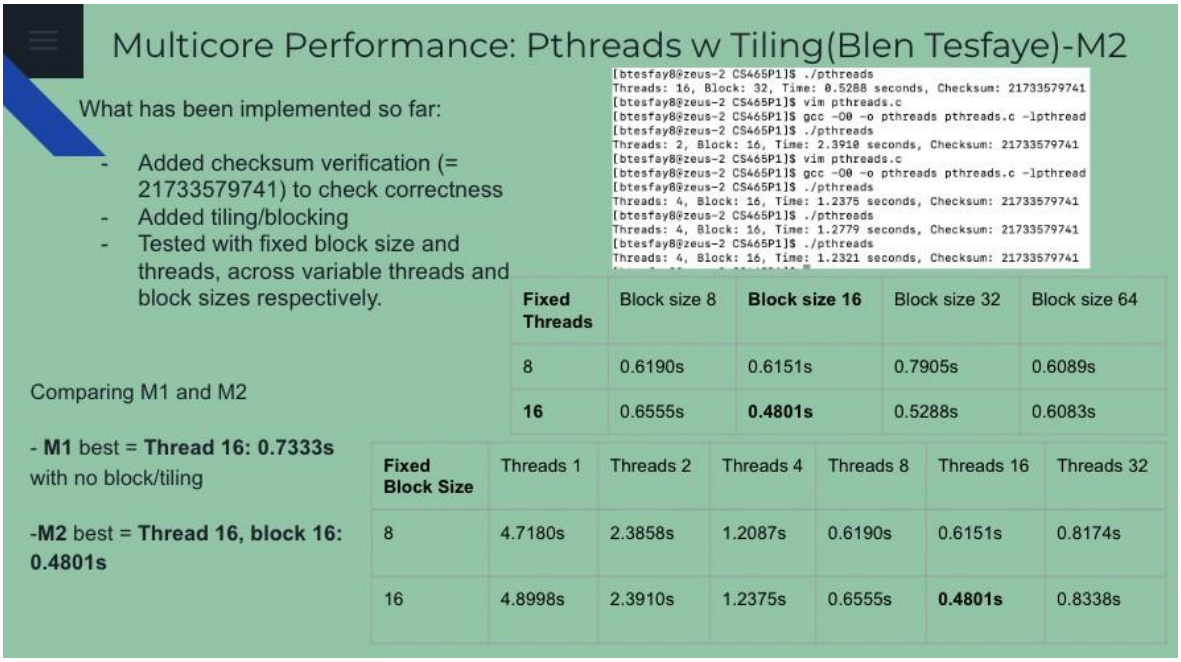
- Comparing our Pthreads results to the other group's, it's a mixed picture. Without tiling, our implementation was actually slightly faster, I got 0.733 seconds with 16 threads, while they got 0.751 seconds. However, once tiling was added, their results were better. Their best result was 0.384 seconds using 16 threads with a tile size of 128, while our best was 0.4801 seconds using 16 threads with block size 16. The main reason they did better with tiling is that they tested a much wider range of tile sizes all the way up to 256, while I only tested up to 64. They found that larger tile sizes like 128 worked better on Zeus, which I didn't discover because I stopped early at 64. They also cached the A[i][k] value into a local variable inside the inner loop, which reduces memory fetches. Overall,

we both used the same core approach of splitting rows across threads, but their more thorough sweep of tile sizes gave them the edge and better results in the tiled version.

Our(Group 7) results:




| Threads | Time R1 | Time R2 | Time R3 |
|---|---|---|---|
| 2 | 3.6321s | 3.7163s | 3.6860s |
| 4 | 1.8655s | 1.8542s | 1.8403s |
| 8 | 0.9561s | 0.9657s | 0.9701s |
| 16 | 0.7259s | 0.7742s | 0.7851s |




| Fixed Threads | Block size 8 | Block size 16 | Block size 32 | Block size 64 |
|---|---|---|---|---|
| 8 | 0.6190s | 0.6151s | 0.7905s | 0.6089s |
| 16 | 0.6555s | 0.4801s | 0.5288s | 0.6083s |

| Fixed Block Size | Threads 1 | Threads 2 | Threads 4 | Threads 8 | Threads 16 | Threads 32 |
|---|---|---|---|---|---|---|
| 8 | 4.7180s | 2.3858s | 1.2087s | 0.6190s | 0.6151s | 0.8174s |
| 16 | 4.8998s | 2.3910s | 1.2375s | 0.6555s | 0.4801s | 0.8338s |

Group 8 results:

## Section 2 - POSIX with no tiling

```
top - 02:33:29 up 5 days,  9:16,  5 users,  load average: 1.06, 0.48, 0.54
Tasks: 625 total,   2 running, 622 sleeping,   1 stopped,   0 zombie
%Cpu(s):  4.2 us,  0.0 sy,  0.0 ni, 95.8 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
MiB Mem :  63627.8 total,  44759.6 free,   4170.3 used,  14697.9 buff/cache
MiB Swap:  32012.0 total,  32001.7 free,     10.2 used.  57203.4 avail Mem
```

| Threads | Run 1 | Run 2 | Run 3 | Average (s) |
|---|---|---|---|---|
| 2 | 3.699099 | 3.456868 | 4.480792 | 3.8789197 |
| 4 | 2.010367 | 1.820807 | 1.840996 | 1.8907233 |
| 8 | 0.950803 | 0.948206 | 0.956142 | 0.951717 |
| 16 | 0.766159 | 0.723415 | 0.762910 | **0.750828** |
| 32 | 1.092059 | 1.008162 | 1.028673 | 1.0429647 |
| 64 | 1.524431 | 1.443865 | 1.482325 | 1.4835403 |
| 128 | 1.472935 | 1.467221 | 1.529831 | 1.4899957 |
| 256 | 1.528058 | 1.492143 | 1.463867 | 1.4946893 |

## Section 2 - POSIX with tiling

| Threads/Tile | 2 | 4 | 8 | 16 | 32 | 64 | 128 | 256 |
|---|---|---|---|---|---|---|---|---|
| 2 | 3.495204 | 2.520403 | 2.226384 | 2.097100 | 2.005886 | 1.960912 | 1.955837 | 1.920074 |
| 4 | 1.776910 | 1.251596 | 1.111522 | 1.053584 | 0.998618 | 0.979168 | 0.975770 | 0.962730 |
| 8 | 0.908208 | 0.645649 | 0.570588 | 0.545644 | 0.519295 | 0.506658 | 0.504958 | 0.493809 |
| 16 | 0.741228 | 0.503845 | 0.452023 | 0.405761 | 0.389625 | 0.384642 | **0.384390** | 0.419923 |
| 32 | 0.963117 | 0.701365 | 0.571664 | 0.599699 | 0.541777 | 0.524317 | 0.553290 | 0.529405 |
| 64 | 1.350199 | 0.951314 | 0.814250 | 0.785412 | 0.733867 | 0.711433 | 0.739303 | 0.689599 |
| 128 | 1.357243 | 0.980261 | 0.865202 | 0.796067 | 0.796374 | 0.782473 | 0.785954 | 0.763278 |
| 256 | 1.381598 | 1.000468 | 0.885794 | 0.801142 | 0.764855 | 0.786194 | 0.793383 | 0.776729 |

Note: 3 runs per partition

**Multicore Performance, OpenMP:**

Comparing Code:

- When comparing my OpenMP code with Group 8's OpenMP code, I notice that when it comes to the OpenMP code without tiling, my program's division of labor per thread differs from the code I saw in Group 8's work. To divide the work between threads in matMul, I divided the first for loop by for(int i = ID * 16; i < (ID + 1) * 16; i++). To explain this, assuming 16 = 1024 / num_threads (64 threads in this case), This divides each

thread(ID number) into its own blocks of 16 spaces and ends its loop when it is done. Group 8's openMP no tiling code shows them using numthreads schedule(static) in order to divide their work, which may have been more effective according to results.

- For OpenMP tiling, I find Group 8's code a tad bit more reliable than my own because the separation of thread labor through using schedule(static) collapse(2) is more trustworthy to provide correct output rather than the separation of thread labor through the calculations that I attempted.

Comparing Results:

- When comparing my results to Group 8's results, I notice that without tiling, which is where I feel the most confident in my code's accuracy and differ more from Group 8's code, our results seem pretty similar on an average basis. Their best accomplished time was using 16 threads and they achieved 0.755 seconds of time. My best result also was using 16 threads and I achieved a time average of 0.737 seconds of time. With tiling, my results seem better than Group 8's results, with my best time being 0.307 seconds at 16 threads using 32 block sized tiles. Group 8 also achieved their best time using 16 threads at a 32 block size, but their time result was over 0.5 seconds which is a solid difference from my results. Results like these can alter this much by either implementation method or testing differences. I believe these time differences come from the openMP itself dividing the thread labor instead of doing it manually/mathematically. The full results of both me (green slides) and Group 8's results (blue slides) are found below.

-

## Multicore Platform exploitation using OpenMP (Natal Sabbar)

### - OpenMP implementation Without Tiling

Everything Done in Zeus-2

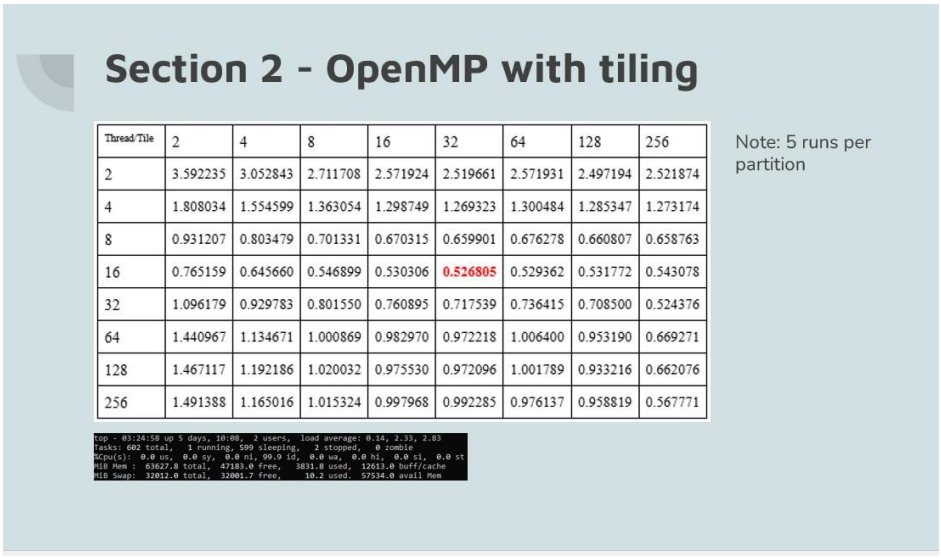
| Thread Count | 4 | 8 | 16 | 32 | 64 |
|---|---|---|---|---|---|
| Avg of 3 results in seconds (s). | 1.833 seconds | 0.942 seconds | 0.737 seconds | 0.941 seconds | 1.287 seconds |

## OpenMP threading WITH Tiling

| Block Sizes<br>\| \| \| \| \|<br>V V V V<br>V | Thread Count<br>-><br>-> -> -> | 4 | 8 | 16 | 32 |
|---|---|---|---|---|---|
| 8 | | 0.905 seconds | 0.469 seconds | 0.368 seconds | 0.526 seconds |
| 16 | | 0.838 seconds | 0.433 seconds | 0.339 seconds | 0.483 seconds |
| 32 | | 0.798 seconds | 0.413 seconds | 0.307 seconds | 0.455 seconds |
| 64 | | 0.825 seconds | 0.425 seconds | 0.332 seconds | 0.487 seconds |

## Section 2 - OpenMP with no tiling

| Threads | Run 1 | Run 2 | Run 3 | Run 4 | Run 5 | Average (s) |
|---|---|---|---|---|---|---|
| 2 | 3.714121 | 3.714363 | 3.663483 | 3.691043 | 3.702818 | 3.697166 |
| 4 | 1.901286 | 1.890763 | 1.884462 | 1.875652 | 1.878544 | 1.886141 |
| 8 | 0.964793 | 0.965491 | 0.971657 | 0.974259 | 0.961752 | 0.967590 |
| 16 | 0.754161 | 0.753914 | 0.742596 | 0.778502 | 0.747575 | **0.755350** |
| 32 | 1.160913 | 1.179817 | 1.183717 | 1.143532 | 1.171110 | 1.167818 |
| 64 | 1.459350 | 1.469663 | 1.435886 | 1.435756 | 1.510435 | 1.462218 |
| 128 | 1.486080 | 1.542693 | 1.548134 | 1.477179 | 1.532347 | 1.517287 |
| 256 | 1.537728 | 1.540407 | 1.492936 | 1.507912 | 1.494868 | 1.514770 |




| Thread/Tile | 2 | 4 | 8 | 16 | 32 | 64 | 128 | 256 |
|---|---|---|---|---|---|---|---|---|
| 2 | 3.592235 | 3.052843 | 2.711708 | 2.571924 | 2.519661 | 2.571931 | 2.497194 | 2.521874 |
| 4 | 1.808034 | 1.554599 | 1.363054 | 1.298749 | 1.269323 | 1.300484 | 1.285347 | 1.273174 |
| 8 | 0.931207 | 0.803479 | 0.701331 | 0.670315 | 0.659901 | 0.676278 | 0.660807 | 0.658763 |
| 16 | 0.765159 | 0.645660 | 0.546899 | 0.530306 | **0.526805** | 0.529362 | 0.531772 | 0.543078 |
| 32 | 1.096179 | 0.929783 | 0.801550 | 0.760895 | 0.717539 | 0.736415 | 0.708500 | 0.524376 |
| 64 | 1.440967 | 1.134671 | 1.000869 | 0.982970 | 0.972218 | 1.006400 | 0.953190 | 0.669271 |
| 128 | 1.467117 | 1.192186 | 1.020032 | 0.975530 | 0.972096 | 1.001789 | 0.933216 | 0.662076 |
| 256 | 1.491388 | 1.165016 | 1.015324 | 0.997968 | 0.992285 | 0.976137 | 0.958819 | 0.567771 |



- -

**Comparison of Memory Hierarchy Optimization (Tiling/Blocking) with Group 8 - Report**

Our implementation focused on memory hierarchy optimization using tiling/blocking techniques for matrix multiplication on Zeus. The goal was to improve cache utilization by breaking the matrix into smaller blocks and processing those blocks sequentially. By operating on smaller chunks of data, the algorithm increases data reuse within the cache and reduces the frequency of costly memory accesses. This approach was implemented using a fixed block size defined at compile time, and performance was evaluated by testing block sizes of 16 and 32 using multiple runs with averaged results.

Group 8 implemented a similar tiling strategy, but extended it by introducing additional flexibility and scalability in their design. While our implementation used fixed block sizes, their code allowed tile size to be specified dynamically at runtime, enabling them to explore a much wider range of configurations. Both implementations use the same fundamental tiling structure, with nested loops iterating over sub-blocks of the matrix, and both aim to improve cache locality by restructuring memory access patterns. As a result, the core memory hierarchy optimization approach is consistent across both implementations.

However, Group 8 enhanced their tiling implementation by integrating parallel execution through OpenMP and POSIX threads. In their OpenMP version, the tiled loops are parallelized using compiler directives, allowing multiple threads to process different portions of the matrix simultaneously. In contrast, our implementation executes the same tiled logic sequentially. This makes the comparison particularly meaningful, as both implementations apply the same memory

optimization technique, but Group 8 further improves performance by leveraging multicore parallelism.

In terms of performance, our results showed that tiling significantly reduced execution time compared to the baseline, with block size 16 achieving the best performance at approximately 4584 ms, representing a speedup of about 1.68×. Group 8 observed similar trends in their results, where optimal performance also occurred at moderate block sizes, typically in the range of 16 to 32. This consistency reinforces the conclusion that properly chosen block sizes are critical for maximizing cache efficiency. However, their runtimes were substantially lower overall, with their OpenMP tiled implementation reaching approximately 526 ms and their POSIX implementation achieving even faster execution at around 384 ms. The primary reason for this difference is that their implementation combines tiling with parallel execution, whereas our implementation isolates the effect of memory hierarchy optimization alone.

Our implementation effectively demonstrates the impact of tiling on cache performance by showing clear improvements over the baseline and identifying an optimal block size range. Group 8's implementation builds on the same foundation but provides a more comprehensive solution by combining memory optimization with parallelism and broader experimentation. While their approach achieves significantly better performance, our implementation provides a focused evaluation of how tiling alone improves execution efficiency through better cache utilization.

This comparison highlights how memory hierarchy optimization alone improves performance, while combining it with parallelism leads to significantly greater overall speedup.

**Their findings for M2:**

## Section 2 - OpenMP with tiling

| Thread/Tile | 2 | 4 | 8 | 16 | 32 | 64 | 128 | 256 |
|---|---|---|---|---|---|---|---|---|
| 2 | 3.592235 | 3.052843 | 2.711708 | 2.571924 | 2.519661 | 2.571931 | 2.497194 | 2.521874 |
| 4 | 1.808034 | 1.554599 | 1.363054 | 1.298749 | 1.269323 | 1.300484 | 1.285347 | 1.273174 |
| 8 | 0.931207 | 0.803479 | 0.701331 | 0.670315 | 0.659901 | 0.676278 | 0.660807 | 0.658763 |
| 16 | 0.765159 | 0.645660 | 0.546899 | 0.530306 | **0.526805** | 0.529362 | 0.531772 | 0.543078 |
| 32 | 1.096179 | 0.929783 | 0.801550 | 0.760895 | 0.717539 | 0.736415 | 0.708500 | 0.524376 |
| 64 | 1.440967 | 1.134671 | 1.000869 | 0.982970 | 0.972218 | 1.006400 | 0.953190 | 0.669271 |
| 128 | 1.467117 | 1.192186 | 1.020032 | 0.975530 | 0.972096 | 1.001789 | 0.933216 | 0.662076 |
| 256 | 1.491388 | 1.165016 | 1.015324 | 0.997968 | 0.992285 | 0.976137 | 0.958819 | 0.567771 |

Note: 5 runs per partition

```
top - 03:24:58 up 5 days, 10:08,  2 users,  load average: 0.14, 2.33, 2.83
Tasks: 602 total,   1 running, 599 sleeping,   2 stopped,   0 zombie
%Cpu(s):  0.0 us,  0.0 sy,  0.0 ni, 99.9 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
MiB Mem :  63627.8 total,  47183.0 free,   3831.8 used,  12613.0 buff/cache
MiB Swap:  32012.0 total,  32001.7 free,     10.2 used.  57534.0 avail Mem
```

## Section 2 - POSIX with tiling

| Threads/Tile | 2 | 4 | 8 | 16 | 32 | 64 | 128 | 256 |
|---|---|---|---|---|---|---|---|---|
| 2 | 3.495204 | 2.520403 | 2.226384 | 2.097100 | 2.005886 | 1.960912 | 1.955837 | 1.920074 |
| 4 | 1.776910 | 1.251596 | 1.111522 | 1.053584 | 0.998618 | 0.979168 | 0.975770 | 0.962730 |
| 8 | 0.908208 | 0.645649 | 0.570588 | 0.545644 | 0.519295 | 0.506658 | 0.504958 | 0.493809 |
| 16 | 0.741228 | 0.503845 | 0.452023 | 0.405761 | 0.389625 | 0.384642 | **0.384390** | 0.419923 |
| 32 | 0.963117 | 0.701365 | 0.571664 | 0.599699 | 0.541777 | 0.524317 | 0.553290 | 0.529405 |
| 64 | 1.350199 | 0.951314 | 0.814250 | 0.785412 | 0.733867 | 0.711433 | 0.739303 | 0.689599 |
| 128 | 1.357243 | 0.980261 | 0.865202 | 0.796067 | 0.796374 | 0.782473 | 0.785954 | 0.763278 |
| 256 | 1.381598 | 1.000468 | 0.885794 | 0.801142 | 0.764855 | 0.786194 | 0.793383 | 0.776729 |

Note: 3 runs per partition

**Our findings for M2:**

# Memory Hierarchy/ Tiling-Blocking ( Andrew Espinosa)

```
[aespin6@zeus-2 CS465]$ gcc -O0 tiled.c -o tiled
[aespin6@zeus-2 CS465]$ ./tiled
Tiled time (BLOCK=16): 4553.134 ms
[aespin6@zeus-2 CS465]$ ./tiled
Tiled time (BLOCK=16): 4567.048 ms
[aespin6@zeus-2 CS465]$ ./tiled
Tiled time (BLOCK=16): 4571.204 ms
[aespin6@zeus-2 CS465]$ ./tiled
Tiled time (BLOCK=16): 4545.647 ms
[aespin6@zeus-2 CS465]$ ./tiled
Tiled time (BLOCK=16): 4681.281 ms
[aespin6@zeus-2 CS465]$
```

Execution on Zeus
(multiple runs each)

**Implementation Summary:**

- Implemented tiled/blocking matrix multiplication
- Tested multiple block sizes (16, 32)
- Ran multiple trials and computed average execution times
- Measured execution time using clock_gettime()
- Compiled with -O0 on Zeus

```
[aespin6@zeus-2 CS465]$ gcc -O0 tiled.c -o tiled
[aespin6@zeus-2 CS465]$ ./tiled
Tiled time (BLOCK=32): 4786.079 ms
[aespin6@zeus-2 CS465]$ ./tiled
Tiled time (BLOCK=32): 4977.740 ms
[aespin6@zeus-2 CS465]$ ./tiled
Tiled time (BLOCK=32): 4825.499 ms
[aespin6@zeus-2 CS465]$ ./tiled
Tiled time (BLOCK=32): 4885.708 ms
[aespin6@zeus-2 CS465]$ ./tiled
Tiled time (BLOCK=32): 4826.530 ms
[aespin6@zeus-2 CS465]$
```

| Version | Block Size | Avg Time (ms) |
|---|---|---|
| **Baseline** | - | **7682** |
| **Tiled** | **16** | **4584** |
| **Tiled** | **32** | **4860** |

**Speedup (Block 16 vs Baseline): ~1.68×**

**Observations**

- Tiling significantly reduced execution time compared to the baseline
- Block size 16 consistently performed better than block size 32
- Averaging results provided more reliable performance measurements

**Progression**

- Initial results were based on single runs (Milestone 1)
- Updated to multiple runs with averaging for improved accuracy (Milestone 2)